TD 3 : Pointeurs, Chaînes de Caractères et Structures

Exercice 1 : Pointeurs et Chaînes de Caractères

Soit le programme suivant :

```
#include <stdio.h>
#include <string.h>

int main() {
    char s[] = "Informatique";
    char *p = s;
    printf("%c\n", *p);
    p += 5;
    printf("%c\n", *p);
    printf("%s\n", p);
    p = s + strlen(s) - 1;
    printf("%c\n", *p);
    return 0;
}
```

1. Qu'affiche ce programme ? Justifiez chaque ligne affichée.

Exercice 2 : Trace d'Exécution avec Chaînes et Pointeurs

Qu'affiche le programme suivant ?

```
#include <stdio.h>
#include <string.h>
void transformer(char *s) {
    int i = 0;
    while (s[i] != '\0') {
        if (s[i] >= 'a' && s[i] <= 'z')
        s[i] = s[i] - 32;
        i++;
    }
}
int main() {
    char mot1[] = "bonjour";
    char mot2[] = "Monde";
    char resultat[20] = "";
    strcat(resultat, mot1);
    strcat(resultat, " ");
    strcat(resultat, mot2);
    printf("%s\n", resultat);
    transformer(resultat);
    printf("%s\n", resultat);

    printf("Longueur : %d\n", strlen(resultat));
    printf("Premier : %c, Dernier : %c\n", resultat[0], resultat[
        strlen(resultat)-1]);
    return 0;
}
```

Exercice 3 : Structures et Pointeurs

On définit la structure suivante représentant un étudiant :

```
typedef struct {
    char nom[50];
    char prenom[50];
    int  age;
    float moyenne;
} Etudiant;
```

1. Déclarez une variable `e1` de type `Etudiant` et initialisez-la avec les valeurs suivantes : nom = `"Alaoui"`, prénom = `"Youssef"`, âge = 20, moyenne = 14.5. Affichez ses champs.
2. Écrivez une fonction `void afficher(Etudiant *e)` qui affiche tous les champs d'un étudiant passé par pointeur.
3. Écrivez une fonction `void augmenterMoyenne(Etudiant *e, float bonus)` qui ajoute un bonus à la moyenne d'un étudiant. Pourquoi faut-il passer `e` par pointeur ?
4. Écrivez une fonction `Etudiant *meilleurEtudiant(Etudiant *tab, int n)` qui retourne un pointeur vers l'étudiant ayant la meilleure moyenne dans un tableau de `n` étudiants.

Exercice 4 : Structures Imbriquées et Allocation Dynamique

On souhaite gérer les étudiants d'une promotion en enrichissant leur représentation. On définit d'abord une structure `Date` pour la date de naissance, puis une structure `Adresse`, toutes deux imbriquées dans une structure `Etudiant` :

```
typedef struct {
    int jour;
    int mois;
    int annee;
} Date;

typedef struct {
    char ville[50];
    char pays[50];
} Adresse;

typedef struct {
    char    nom[50];
    char    prenom[50];
    Date    dateNaissance;
    Adresse adresse;
    float   moyenne;
} Etudiant;
```

1. Déclarez et initialisez une variable `e1` de type `Etudiant` avec les valeurs suivantes : nom = `"Benali"`, prénom = `"Sara"`, date de naissance = 12/03/2004, ville = `"Marrakech"`, pays = `"Maroc"`, moyenne = 15.75. Affichez tous ses champs en utilisant l'opérateur `.` et l'opérateur `->`.

2. Écrivez une fonction `void afficherEtudiant(Etudiant *e)` qui affiche tous les champs d'un étudiant (y compris la date de naissance et l'adresse) via un pointeur.
3. Écrivez une fonction `int memeVille(Etudiant *e1, Etudiant *e2)` qui retourne 1 si les deux étudiants habitent la même ville, 0 sinon. Utilisez `strcmp` pour comparer les champs `adresse.ville`.
4. Écrivez un programme complet qui :
   (a) Demande à l'utilisateur le nombre d'étudiants `n`.
   (b) Alloue dynamiquement un tableau de `n` structures `Etudiant`.
   (c) Saisit les informations de chaque étudiant (nom, prénom, date de naissance, ville, pays, moyenne).
   (d) Affiche la liste complète des étudiants à l'aide de la fonction `afficherEtudiant`.
   (e) Affiche le nom et la moyenne de l'étudiant ayant la meilleure moyenne.
   (f) Compte et affiche le nombre de paires d'étudiants habitant la même ville.
   (g) Libère la mémoire allouée.
5. Expliquez la différence d'accès aux champs imbriqués entre une variable directe et un pointeur. Complétez le tableau suivant :

Champ	**Via variable e**	**Via pointeur p**
Nom	`e.nom`	
Jour de naissance	`e.dateNaissance.jour`	
Ville	`e.adresse.ville`	
Moyenne		`p->moyenne`